Proven both ways across.

Every Portaldot bridge so far trusts someone on the way back. Backhaul trusts no one, in either direction.

Portaldot V3 testnet ↔ BNB Chain testnet · 0 trusted parties, either way

backhaul-bridge.vercel.app



BNB Chain · a header every 0.45 s

6 of 9 seals in · 2 of 2 signatures back

Portaldot · a block every 6 s

A bridge is only as safe as the few keys that can sign for it.

Portaldot ↔ BNB Chain today

Custodial. You send to an address, and a service pays you out on the other side.

A committee on the way back

The same risk as a multisig: whoever holds a few keys can sign for everything locked.

$806M

taken from three bridges that a handful of keys could sign for

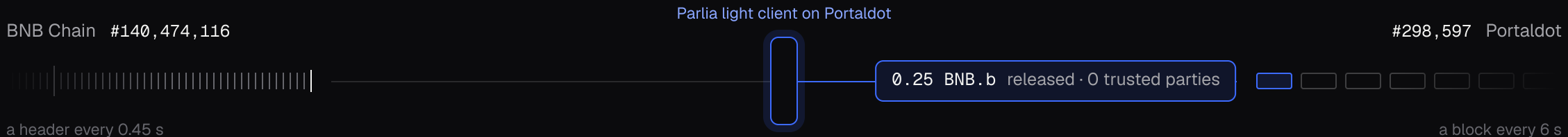
Ronin	5 of 9 validator keys	$625M
Harmony Horizon	2 of 5 multisig keys	~$100M
Orbit Chain	7 of 10 multisig keys	$81.5M

Sources: Ronin post-mortem (Mar 2022) · Elliptic (Jun 2022) · Blockworks (Jan 2024)

Each chain runs a light client of the other. Nothing crosses without a proof.

0 trusted parties, either way

BNB Chain → Portaldot



6 / 6 header #140,474,079 is final · sealed through #140,474,112

0	1	2	3	4	5	6	7	8
0826	1a3d	3241	5338	76d7	7984	7f5f	d447	f9a1

- Sealed by 6 of 9 · #140,474,079
- Receipt proof verified · Locked log
- Released on Portaldot #298,597

• replay of real headers · BNB Chain testnet #140,474,047–#140,474,446 · Portaldot V3 #298,580–#298,620 · recorded 03 Nov 2026 00:02 UTC

Six of nine BNB Chain validators seal it. Portaldot checks every seal itself.

6 / 9

distinct validators seal the headers above your lock

≈ 44 headers · ≈ 20 s



Lock

1

on BNB Chain

You lock tBNB in the bridge contract on BNB Chain.

`BscGateway.lockBNB`

Seal

2

6 of 9 validators

A Parlia light client on Portaldot's EVM checks each seal against the BNB Chain validator set it has proven.

`ParliaClient.submitWindow`

Prove

3

receipt proof

A Merkle-Patricia proof shows your lock's receipt under the finalized header's receipts root.

`BscVerifier · Locked log`

Release

4

on Portaldot

BNB.b arrives as a native pallet-assets asset, with a little tPOT so a fresh wallet can move it.

`BNB.b · pallet-assets`

Portaldot's own BEEFY finality, checked signature by signature on BNB Chain.

2/2

BEEFY authorities sign every commitment

`set 520`



signed MMR root	`0x41f9…4038`
header, BLAKE2b	`#309,018`
revive child trie	`0x2ced…3eb8`

Lock 1

on Portaldot

You lock tPOT or BNB.b, and the outbox writes a commitment to it in revive storage.

`PortaldotGateway · Outbox`

Sign 2

2 of 2 BEEFY authorities

Both authorities sign the MMR root. A light client on BNB Chain recovers each secp256k1 signature.

`PortaldotLightClient`

Prove 3

MMR + storage proof

An MMR proof ties your block to the signed root. A BLAKE2b child-trie proof reads the commitment.

`PortaldotVerifier · 3.3 KB proof`

Release 4

on BNB Chain

pPOT is minted or tBNB released to you, once per nonce.

`BscGateway.finalize`

Every part of Backhaul is a Portaldot V3 primitive.

Portaldot V3 runtime

pallet-beefy · pallet-mmr

Bridge-grade finality, built in

Portaldot signs an MMR root of its blocks with secp256k1 keys any EVM chain can check. Backhaul is the first bridge to verify it.

78k gas per signed commitment

pallet-revive

Unmodified Solidity next to the pallets

The Parlia light client and the gateways are ordinary Solidity on Portaldot's EVM, and their storage is provable from the state root.

ParliaClient · BscVerifier · gateways

pallet-assets · ERC-20 precompile · AssetConversion

A native asset, not a wrapped token

BNB.b lives in pallet-assets. SubWallet shows it, MetaMask sees it through the precompile, and it can trade in the runtime's own pools.

fixed supply, every role held by the bridge

pallet-contracts · ink!

The same light client on the second VM

An ink! twin of the Parlia client gives WASM contracts proven BNB Chain state, measured side by side with the EVM build.

20 KB WASM · 48-header window in 12.32 ms

Modular runtime, two VMs, native assets and its own bridge finality: Portaldot V3 ships all of it today.

Paid in seconds. Repaid only after the proof.

seconds

the proof settles behind it

Order opened

Your funds go into escrow on the origin chain as an ERC-7683 order.

Solver pays you

An independent solver pays you on the destination chain, out of its own funds.

Fill proven

The light client checks the solver's fill on the other chain.

Solver repaid

Only now does your escrow move, to the solver.

ERC-7683 orders

A standard intent format, so any solver can read an order and fill it.

Proof-gated LP vaults

Vaults fill only against a proof that the order was opened. No operator decides what they pay out.

Fees to LPs

Every fill pays a spread to whoever supplied the liquidity. That is the business model.

Nobody fills before the deadline? A proof that the order was never filled refunds you.

Tested on real data from both chains. Then attacked.

434

tests passing

`forge 263 · sdk 92 · relayer 44 · ink! 35`

520 → 521

live BEEFY set handoff, verified on a BNB Chain testnet fork

`commitment 77,328 gas · header 76,296 gas`

44

real BNB Chain headers finalized as one window

`39 KB · 7 sealers · 8 seals from outside the set not counted`

12 / 12

forged proofs rejected by the attack lab

`11 of them again in the live-data rehearsals`

Gas per call on the BNB Chain side



On testnet · 10 contracts

Portaldot	PortaldotGateway	`0x4a83…23F3`
Portaldot	Outbox	`0x788e…09d5`
BNB Chain	PortaldotLightClient	`0x4a83…23F3`
BNB Chain	BscGateway	`0xb05f…daeB`
Portaldot	canonical · locked	`0xddc0…fb3d`
BNB Chain	canonical · beefy	`0x9b9c…fb3c`
BNB Chain	canonical · beefy	`0x054d…88d4`

Every number on this slide is generated from results/ and deployments/ in the repository.

A route for BNB Chain liquidity into Portaldot, with no custodian.

Who uses it

POT holders

moving between BNB Chain and V3 mainnet without handing coins to a service.

BNB Chain liquidity

arriving as native BNB.b, ready for Portaldot's own AssetConversion pools.

Portaldot dApps

reading proven BNB Chain data, starting with the Chainlink BNB/USD feed (BscFeed).

How it earns

Fast fills

a spread on every fill, earned by whoever supplies the liquidity: an LP vault or an independent solver.

Canonical transfers

a 0.1% fee on tPOT sent out, which today pays the gas a fresh wallet needs when BNB.b arrives.

No middleman

fees reach LPs by contract. Nobody can redirect them or freeze a transfer.

Path to mainnet

- **Testnet** live
  both light clients, bridges, settlers and vaults
- **Audit**
  light clients, the Merkle trie and BLAKE2 code
- **V3 mainnet, with caps**
  per-transfer and total limits that rise as it proves itself

`full deploy: 3,112 tPOT + 0.0012 tBNB`

BNB Chain is the first counterparty, not the last.

Mainnet light clients for V3 Live

01

The same immutable contracts, started from a mainnet checkpoint, with caps that rise as they prove themselves.

More chains on the same verifiers

02

The BEEFY light client runs on any EVM chain with the BLAKE2 precompile, so each new chain is a deploy, not a rewrite.

An SDK and a relayer anyone can run

03

Proof builders for BEEFY, MMR, Substrate tries, Parlia and receipts. A relayer can delay a transfer, never forge one.

Findings filed for the platform

04

Public nodes run without off-chain indexing, so mmr_generateProof fails, and pallet-revive's Ethereum blocks report stateRoot equal to transactionsRoot.

Continued support pays for the audit, a capped mainnet launch, and relayers that run for everyone.

Bridge both ways. Then try to break it.

Every Portaldot bridge so far trusts someone on the way back. Backhaul trusts no one, in either direction.

App	backhaul-bridge.vercel.app/app ↗
Attack lab	backhaul-bridge.vercel.app/attacks ↗
Source	github.com/RaYYeR220/backhaul ↗
Docs	github.com/RaYYeR220/backhaul/tree/main/docs ↗

MIT licensed · Portaldot V3 testnet ↔ BNB Chain testnet